

Making quotient polynomial

Smoked Perplexity.ai

This doc may have high perplexity. Not checked for values mismatch.

1 Introduction

This document provides a comprehensive analysis of the PLONK protocol's quotient polynomial construction and the complete set of polynomial commitments required for proving circuit satisfiability. We examine the protocol using the concrete non-linear circuit example:

$$z = x^2 + xy + 3$$

with specific values $x = 2, y = 5$, resulting in $z = 17$.

2 Circuit Decomposition and Gate Constraints

2.1 Gate Structure

Our example circuit is decomposed into three gates:

$$\text{Gate 1: } a = x \cdot x \quad (1)$$

$$\text{Gate 2: } b = x \cdot y \quad (2)$$

$$\text{Gate 3: } z = a + b + 3 \quad (3)$$

2.2 Witness Assignment Table

The witness values for our circuit are organized as follows:

Gate	L	R	O	Operation	Constraint
1	2	2	4	$a = x \cdot x$	$L \cdot R = O$
2	2	5	10	$b = x \cdot y$	$L \cdot R = O$
3	4	10	17	$z = a + b + 3$	$L + R + 3 = O$

3 Polynomial Construction

3.1 Wire Assignment Polynomials

The wire polynomials $L(X)$, $R(X)$, and $O(X)$ are constructed by interpolating the witness values over the domain $H = \{\omega^0, \omega^1, \omega^2\}$ where ω is a primitive 3rd root of unity.

For each polynomial:

$$L(X) = 2 \cdot \ell_1(X) + 2 \cdot \ell_2(X) + 4 \cdot \ell_3(X) \quad (4)$$

$$R(X) = 2 \cdot \ell_1(X) + 5 \cdot \ell_2(X) + 10 \cdot \ell_3(X) \quad (5)$$

$$O(X) = 4 \cdot \ell_1(X) + 10 \cdot \ell_2(X) + 17 \cdot \ell_3(X) \quad (6)$$

where $\ell_i(X)$ are Lagrange basis polynomials for the domain H .

3.2 Selector Polynomials

The general PLONK gate constraint is:

$$q_L(X)L(X) + q_R(X)R(X) + q_M(X)L(X)R(X) + q_O(X)O(X) + q_C(X) = 0$$

For our circuit, the selector values are:

Gate	q_L	q_R	q_M	q_O	q_C
1	0	0	1	-1	0
2	0	0	1	-1	0
3	1	1	0	-1	3

Each selector is interpolated into a polynomial over the same domain H .

3.3 Permutation Polynomials

The permutation polynomials $S_L(X)$, $S_R(X)$, and $S_O(X)$ encode the global variable consistency constraints. For our example:

Variable Occurrences:

- Variable x appears at wire positions: (1,L), (1,R), (2,L)
- Variable a appears at: (1,O), (3,L)
- Variable b appears at: (2,O), (3,R)
- Variables y and z appear only once each

Permutation Mapping: The permutation σ creates cycles for repeated variables:

$$\sigma : (1, L) \rightarrow (1, R) \rightarrow (2, L) \rightarrow (1, L), \quad (1, O) \leftrightarrow (3, L), \quad (2, O) \leftrightarrow (3, R)$$

3.4 Grand Product Polynomial

The grand product polynomial $Z(X)$ is constructed to prove the permutation argument. It satisfies:

$$Z(\omega^0) = 1 \tag{7}$$

$$Z(\omega^{i+1}) = Z(\omega^i) \cdot \frac{(L(\omega^i) + \beta\omega^i + \gamma)(R(\omega^i) + \beta k_1\omega^i + \gamma)(O(\omega^i) + \beta k_2\omega^i + \gamma)}{(L(\omega^i) + \beta S_L(\omega^i) + \gamma)(R(\omega^i) + \beta S_R(\omega^i) + \gamma)(O(\omega^i) + \beta S_O(\omega^i) + \gamma)} \tag{8}$$

for challenges β, γ and constants k_1, k_2 .

4 Quotient Polynomial Construction

4.1 Constraint Aggregation

The quotient polynomial $T(X)$ aggregates all circuit constraints into a single polynomial divisibility check. It is defined as:

$$T(X) = \frac{\text{GateConstraints}(X) + \alpha \cdot \text{PermutationConstraints}(X) + \alpha^2 \cdot \text{PublicInputConstraints}(X)}{Z_H(X)}$$

where $Z_H(X) = X^n - 1$ is the vanishing polynomial and α is a challenge scalar.

4.2 Gate Constraint Component

The gate constraint polynomial is:

$$\text{GateConstraints}(X) = q_L(X)L(X) + q_R(X)R(X) + q_M(X)L(X)R(X) + q_O(X)O(X) + q_C(X)$$

For our example circuit with correct witness values, this polynomial evaluates to zero at every $\omega^i \in H$:

$$\text{GateConstraints}(\omega^0) = 0 \cdot 2 + 0 \cdot 2 + 1 \cdot (2 \cdot 2) + (-1) \cdot 4 + 0 = 0 \quad (9)$$

$$\text{GateConstraints}(\omega^1) = 0 \cdot 2 + 0 \cdot 5 + 1 \cdot (2 \cdot 5) + (-1) \cdot 10 + 0 = 0 \quad (10)$$

$$\text{GateConstraints}(\omega^2) = 1 \cdot 4 + 1 \cdot 10 + 0 \cdot (4 \cdot 10) + (-1) \cdot 17 + 3 = 0 \quad (11)$$

4.3 Permutation Constraint Component

The permutation constraint ensures variable consistency across gates:

$$\text{PermutationConstraints}(X) = \left(\prod_i \text{numerator}(X) - \prod_i \text{denominator}(X) \right) \cdot Z(X)$$

This component also vanishes on the domain H when the permutation argument is satisfied.

4.4 Complete Quotient Polynomial

For our example, since all constraints are satisfied:

$$T(X) = \frac{0 + \alpha \cdot 0 + \alpha^2 \cdot 0}{Z_H(X)} = 0$$

In general, $T(X)$ is a low-degree polynomial that encodes the aggregated constraint violations.

5 Polynomial Commitments in PLONK

5.1 Commitment Scheme Overview

PLONK uses the KZG polynomial commitment scheme, which provides:

- **Binding:** The prover cannot change the committed polynomial
- **Hiding:** The commitment reveals no information about the polynomial (with blinding)
- **Homomorphic:** Supports polynomial arithmetic on commitments

5.2 Complete List of Committed Polynomials

The PLONK prover creates commitments to the following polynomials:

5.2.1 Witness Polynomials (Round 1)

- $\text{COM}(L(X))$ - Left wire assignments
- $\text{COM}(R(X))$ - Right wire assignments
- $\text{COM}(O(X))$ - Output wire assignments

5.2.2 Permutation Polynomial (Round 2)

- $\text{COM}(Z(X))$ - Grand product polynomial for permutation argument

5.2.3 Quotient Polynomial (Round 3)

- $\text{COM}(T_{lo}(X))$ - Low-degree part of quotient polynomial
- $\text{COM}(T_{mid}(X))$ - Middle-degree part of quotient polynomial
- $\text{COM}(T_{hi}(X))$ - High-degree part of quotient polynomial

The quotient polynomial is split into three parts of degree $< n$ to manage the overall degree bound.

5.3 Blinding for Zero Knowledge

To achieve zero-knowledge, each committed polynomial is blinded by adding random multiples of the vanishing polynomial:

$$\text{Blinded}(f(X)) = f(X) + r_1 \cdot Z_H(X) + r_2 \cdot X \cdot Z_H(X) + \dots$$

where r_i are random blinding factors.

5.4 Preprocessed Commitments (Setup Phase)

The following polynomials are committed during the trusted setup:

- $\text{COM}(q_L(X)), \text{COM}(q_R(X)), \text{COM}(q_M(X)), \text{COM}(q_O(X)), \text{COM}(q_C(X))$ - Selector polynomials
- $\text{COM}(S_L(X)), \text{COM}(S_R(X)), \text{COM}(S_O(X))$ - Permutation polynomials

6 Protocol Flow and Commitment Verification

6.1 Prover Algorithm

1. **Round 1:** Compute and commit to witness polynomials $L(X), R(X), O(X)$
2. **Round 2:** Receive challenges β, γ , compute grand product $Z(X)$, commit to $Z(X)$
3. **Round 3:** Receive challenge α , compute quotient polynomial $T(X)$, split and commit to $T_{lo}(X), T_{mid}(X), T_{hi}(X)$
4. **Round 4:** Receive evaluation challenge ζ , provide polynomial evaluations and opening proofs

6.2 Verifier Algorithm

The verifier checks:

1. All polynomial commitments are valid
2. Polynomial evaluations are consistent with commitments

3. Gate constraints are satisfied: $q_L(\zeta)L(\zeta) + q_R(\zeta)R(\zeta) + q_M(\zeta)L(\zeta)R(\zeta) + q_O(\zeta)O(\zeta) + q_C(\zeta) = 0$
4. Permutation constraint is satisfied
5. Quotient polynomial relation holds: $T(\zeta) \cdot Z_H(\zeta) = \text{LinearizedConstraints}(\zeta)$

7 Security Properties

7.1 Completeness

For any valid witness, the honest prover can construct polynomials that satisfy all constraints, resulting in a quotient polynomial that is well-defined and has the correct degree bound.

7.2 Knowledge Soundness

If the prover can construct a valid proof, then with high probability they know a witness that satisfies the circuit. This follows from the binding property of the polynomial commitment scheme and the soundness of the underlying polynomial protocols.

7.3 Zero Knowledge

The blinded polynomial commitments and evaluations reveal no information about the witness beyond what is implied by the public statement.

8 Conclusion

The PLONK protocol achieves succinct proof generation through polynomial commitments and constraint aggregation. The quotient polynomial serves as the central mechanism for batching all circuit constraints into a single divisibility check, while the comprehensive commitment scheme ensures the integrity of all polynomial relations. The concrete example demonstrates how these abstract concepts translate into practical computations for circuit satisfiability proofs.